

Методичка 10. Работа с файлами

1. Зачем нужна эта тема

Файлы нужны, чтобы программа могла сохранять данные и читать их позже.

Пока программа работает только с переменными, все данные исчезают после завершения программы. Например, если пользователь ввёл список слов, оценки или результаты игры, после закрытия программы эти данные пропадут.

Файлы позволяют:

- читать текст из файла;
- записывать результат в файл;
- хранить настройки программы;
- сохранять список данных;
- обрабатывать входные и выходные данные в задачах;
- делать программы, которые не теряют информацию после завершения.

Примеры использования:

- программа читает стихотворение из файла и считает количество букв;
- программа читает числа из файла и считает сумму;
- программа сохраняет результат работы в `result.txt`;
- программа хранит список товаров или телефонов в текстовом файле.

Эта тема связана со строками, списками, циклами и функциями. Файл обычно читается как строка или список строк, а затем данные обрабатываются уже знакомыми инструментами.

2. Главная идея темы

Файл — это место на компьютере, где можно хранить данные.

Программа может открыть файл, прочитать из него данные или записать данные в него.

Файл — объект на компьютере, в котором хранится информация.

Путь к файлу — место, где находится файл.

Режим открытия файла — способ работы с файлом: чтение, запись, добавление.

Чтение файла — получение данных из файла в программу.

Запись файла — сохранение данных из программы в файл.

Кодировка — правило, по которому символы превращаются в данные файла. Для русского текста обычно удобно использовать `utf-8`.

3. Базовый синтаксис

Открытие файла для чтения

```
file = open("input.txt", "r", encoding="utf-8")

text = file.read()

file.close()

print(text)
```

Что происходит:

- `open()` открывает файл;
- `"input.txt"` — имя файла;
- `"r"` означает режим чтения;
- `encoding="utf-8"` помогает корректно читать русский текст;
- `read()` читает весь файл целиком;
- `close()` закрывает файл.

На что обратить внимание:

- файл должен существовать;
- имя файла нужно писать в кавычках;
- файл лучше закрывать после работы.

Открытие файла через with

```
with open("input.txt", "r", encoding="utf-8") as file:
    text = file.read()

print(text)
```

Такой способ удобнее, потому что файл закрывается автоматически.

Запись в файл

```
with open("result.txt", "w", encoding="utf-8") as file:
    file.write("Привет, файл!")
```

Режим `"w"` означает запись.

Важно: если файл уже был, старое содержимое будет заменено новым.

Добавление в файл

```
with open("result.txt", "a", encoding="utf-8") as file:
    file.write("\nНовая строка")
```

Режим `"a"` означает добавление в конец файла.

Чтение строк файла

```
with open("input.txt", "r", encoding="utf-8") as file:
    lines = file.readlines()

print(lines)
```

`readlines()` читает файл как список строк.

4. Разбор темы от простого к сложному

Шаг 1. Прочитать весь текст из файла

```
with open("text.txt", "r", encoding="utf-8") as file:
    text = file.read()

print(text)
```

Программа открывает файл, читает весь текст и выводит его на экран.

Такой вариант удобен, когда нужно обработать весь текст целиком.

Шаг 2. Посчитать длину текста

```
with open("text.txt", "r", encoding="utf-8") as file:
    text = file.read()

print("Количество символов:", len(text))
```

Здесь файл читается как обычная строка. После чтения можно использовать `len()`.

Шаг 3. Посчитать количество букв

```
with open("text.txt", "r", encoding="utf-8") as file:
    text = file.read()

count = 0

for symbol in text:
    if symbol == "a":
        count = count + 1

print("Количество букв a:", count)
```

Файл прочитан как строка, а затем строка перебирается циклом.

Шаг 4. Записать результат в файл

```
count = 15

with open("result.txt", "w", encoding="utf-8") as file:
    file.write("Количество букв: " + str(count))
```

Метод `write()` записывает строку.

Если нужно записать число, его нужно сначала превратить в строку через `str()`.

Шаг 5. Прочитать файл построчно

```
with open("input.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line)
```

Так можно обрабатывать файл строка за строкой.

Это удобно, если файл большой или если каждая строка — отдельная запись.

Шаг 6. Убрать перенос строки

```
with open("input.txt", "r", encoding="utf-8") as file:
    for line in file:
        line = line.strip()
        print(line)
```

Метод `strip()` убирает лишние пробелы и символ переноса строки в конце.

Шаг 7. Прочитать числа из файла

Пусть в файле `numbers.txt` написано:

```
10
20
30
```

Код:

```
total = 0

with open("numbers.txt", "r", encoding="utf-8") as file:
    for line in file:
        number = int(line)
        total = total + number

print("Сумма:", total)
```

Каждая строка читается как текст. Чтобы сложить числа, нужно использовать `int()`.

Шаг 8. Записать список в файл

```
names = ["Аня", "Борис", "Вика"]

with open("names.txt", "w", encoding="utf-8") as file:
    for name in names:
        file.write(name + "\n")
```

`\n` нужен, чтобы каждое имя записалось на новой строке.

Шаг 9. Прочитать список из файла

```
names = []

with open("names.txt", "r", encoding="utf-8") as file:
    for line in file:
        name = line.strip()
        names.append(name)

print(names)
```

Файл можно превратить в список строк.

5. Частые ошибки учеников

Ошибка 1. Файл не найден

Неправильно:

```
with open("data.txt", "r", encoding="utf-8") as file:
    text = file.read()
```

Почему это ошибка:

Если файла `data.txt` нет в нужной папке, программа упадёт.

Правильно:

```
with open("input.txt", "r", encoding="utf-8") as file:
    text = file.read()
```

Нужно убедиться, что файл существует и находится рядом с программой или указан правильный путь.

Ошибка 2. Забыли указать кодировку

Неправильно:

```
with open("text.txt", "r") as file:
    text = file.read()
```

Почему это может быть проблемой:

Русские буквы могут прочитаться неправильно на некоторых компьютерах.

Правильно:

```
with open("text.txt", "r", encoding="utf-8") as file:
    text = file.read()
```

Ошибка 3. Пытаются записать число напрямую

Неправильно:

```
count = 10

with open("result.txt", "w", encoding="utf-8") as file:
    file.write(count)
```

Почему это ошибка:

`write()` записывает строку, а `count` — число.

Правильно:

```
count = 10

with open("result.txt", "w", encoding="utf-8") as file:
    file.write(str(count))
```

Ошибка 4. Используют режим `"w"` и случайно стирают файл

Неправильно:

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Новая заметка")
```

Почему это может быть ошибкой:

Режим `"w"` перезаписывает файл полностью.

Если нужно добавить данные в конец, нужен режим `"a"`.

Правильно:

```
with open("notes.txt", "a", encoding="utf-8") as file:
    file.write("\nНовая заметка")
```

Ошибка 5. Забывают убрать перенос строки

Неправильно:

```
names = []

with open("names.txt", "r", encoding="utf-8") as file:
    for line in file:
        names.append(line)

print(names)
```

Почему это неудобно:

В строках могут остаться символы `\n`.

Правильно:

```
names = []

with open("names.txt", "r", encoding="utf-8") as file:
    for line in file:
        names.append(line.strip())

print(names)
```

Ошибка 6. Читают файл несколько раз подряд без необходимости

Неправильно:

```
with open("text.txt", "r", encoding="utf-8") as file:
    print(file.read())
    print(file.read())
```

Почему это ошибка:

После первого `read()` файл уже прочитан до конца. Второй `read()` ничего не покажет.

Правильно:

```
with open("text.txt", "r", encoding="utf-8") as file:
    text = file.read()

print(text)
print(text)
```

Ошибка 7. Путают чтение и запись

Неправильно:

```
with open("result.txt", "r", encoding="utf-8") as file:  
    file.write("Ответ")
```

Почему это ошибка:

Режим `"r"` открывает файл для чтения, а не для записи.

Правильно:

```
with open("result.txt", "w", encoding="utf-8") as file:  
    file.write("Ответ")
```

6. Мини-примеры для закрепления

Пример 1. Прочитать файл

```
with open("input.txt", "r", encoding="utf-8") as file:  
    text = file.read()  
  
print(text)
```

Программа читает весь файл и выводит текст. Закрепляется `open`, `read`.

Пример 2. Записать строку

```
with open("result.txt", "w", encoding="utf-8") as file:  
    file.write("Готово")
```

Программа создаёт файл или перезаписывает существующий. Закрепляется режим `"w"`.

Пример 3. Добавить строку

```
with open("log.txt", "a", encoding="utf-8") as file:  
    file.write("\nНовая запись")
```

Программа добавляет данные в конец файла. Закрепляется режим `"a"`.

Пример 4. Посчитать символы

```
with open("text.txt", "r", encoding="utf-8") as file:
    text = file.read()

print(len(text))
```

Закрепляется работа с файлом как со строкой.

Пример 5. Посчитать строки

```
count = 0

with open("input.txt", "r", encoding="utf-8") as file:
    for line in file:
        count = count + 1

print("Строк:", count)
```

Закрепляется построчное чтение.

Пример 6. Прочитать числа

```
numbers = []

with open("numbers.txt", "r", encoding="utf-8") as file:
    for line in file:
        numbers.append(int(line))

print(numbers)
```

Закрепляется преобразование строк из файла в числа.

Пример 7. Записать список

```
items = ["хлеб", "молоко", "сыр"]

with open("items.txt", "w", encoding="utf-8") as file:
    for item in items:
        file.write(item + "\n")
```

Закрепляется запись нескольких строк.

7. Практические задания

Лёгкий уровень

1. Создать файл `hello.txt` и записать туда строку `"Привет"`.
2. Прочитать файл `hello.txt` и вывести его содержимое.
3. Записать в файл три любимых блюда, каждое с новой строки.
4. Прочитать файл со списком имён и вывести каждое имя.
5. Прочитать текст из файла и вывести количество символов.

Средний уровень

1. В файле записаны числа, каждое с новой строки. Посчитать их сумму.
2. В файле записан текст. Посчитать количество букв `"a"`.
3. Пользователь вводит 5 слов. Записать их в файл.
4. Прочитать список слов из файла и сохранить их в список Python.
5. Прочитать файл и вывести только непустые строки.

Повышенный уровень

1. В файле записаны числа. Найти максимальное число.
2. В файле записаны слова. Посчитать количество слов длиннее 5 символов.
3. Сделать программу, которая ведёт простой журнал: пользователь вводит записи, а программа добавляет их в файл.

8. Контрольные вопросы

1. Для чего нужны файлы?
2. Что делает `open()` ?
3. Что означает режим `"r"` ?
4. Что означает режим `"w"` ?
5. Что означает режим `"a"` ?
6. Чем опасен режим `"w"` ?
7. Что делает `read()` ?
8. Что делает `readlines()` ?
9. Зачем нужен `encoding="utf-8"` ?
10. Почему число перед записью нужно преобразовать в строку?
11. Зачем использовать `strip()` при чтении строк?
12. Почему удобно использовать `with open(...)` ?

9. Краткий конспект в конце

Главная идея темы: файлы позволяют программе сохранять данные и читать их

позже.

Основные конструкции:

```
with open("input.txt", "r", encoding="utf-8") as file:  
    text = file.read()
```

```
with open("result.txt", "w", encoding="utf-8") as file:  
    file.write("Ответ")
```

```
with open("log.txt", "a", encoding="utf-8") as file:  
    file.write("\nНовая запись")
```

Важно запомнить:

- `"r"` — чтение;
- `"w"` — запись с перезаписью файла;
- `"a"` — добавление в конец;
- `read()` читает весь файл;
- файл можно читать построчно;
- `write()` записывает только строки;
- `strip()` убирает лишние пробелы и перенос строки;
- `with open(...)` автоматически закрывает файл.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- открывать файл для чтения;
- читать весь файл целиком;
- читать файл построчно;
- открывать файл для записи;
- добавлять данные в конец файла;
- записывать строки в файл;
- преобразовывать данные из файла в числа;
- сохранять список строк в файл;
- читать список строк из файла;
- находить типичные ошибки при работе с файлами.